

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский ядерный университет «МИФИ»



**Институт
интеллектуальных кибернетических систем
Кафедра №22 «Кибернетика»**

Система автоматизированного развёртывания окружений

План управления конфигурацией

Выполнили:

Кузнецова А. А.

Куренный Г. И.

Левченко А. А.

Сысоев К. В.

Чен П. Я.

Б20-504

Деревянко А.К.

Б20-564

Проверяющий:

Козырев В. П.

Содержание

План управления конфигурацией	3
1. Назначение раздела	3
2. Цели процесса УКПО	3
3. Записи УК ПО	4
4. Мероприятия УК ПО	4
4.1. Определение конфигурации	5
4.1.1. Единица конфигурации и их идентификация	5
4.1.2. Контроль среды ЖЦ ПО	5
4.1.2.1 Создание запроса на слияние(merge request)	5
4.1.2.2 Создание документа	5
4.2. Архивирование и воспроизведение	6
4.2.1. Хранение	6
4.2.2 Воспроизведение	6
4.3. Базовая версия	6
4.3.1. Процесс сборки базовой версии	6
4.4. Регистрация проблем	7
4.5. Запрос на изменение	8
4.6. Рассмотрение изменений	10

План управления конфигурацией

Составляющие части единицы конфигурации - требования к ПО, код, документация к ПО, стандарт на разработку, документ о проблемах, готовая сборка.

1. Назначение раздела

В Плане Управления Конфигурацией ПО устанавливаются методы, которые предполагается использовать на протяжении ЖЦ ПО для достижения целей процесса УКПО.

2. Цели процесса УКПО

1. Каждая конфигурации и ее последовательные версии идентифицируются однозначным образом, чтобы сформировать основу для контроля и ссылок на конфигурации.
2. Базовые версии используются в процессах ЖЦ ПО для управления и трассировки конфигураций. Они предоставляют возможность ссылаться на эти конфигурации, контролировать их и отслеживать взаимосвязи между ними.
3. Процесс регистрации проблем важен для выявления несоответствия процессов планам и стандартам создания ПО. Он также помогает обнаружить недостатки в выходных данных процессов ЖЦ ПО и ненормальное поведение программных продуктов. Регистрация проблем гарантирует, что эти проблемы будут решены и не останутся незамеченными.
4. Управление изменениями важно для контроля и утверждения изменений, которые происходят в течение всего ЖЦ ПО. Это включает регистрацию, оценку и анализ этих изменений.
5. Рассмотрение изменений важно для обеспечения эффективного использования методов регистрации проблем и управления изменениями, которые были определены в процессе планирования ПО. Через рассмотрение проблем изменения будут оцениваться, утверждаться или отклоняться.
6. Учет состояния предоставляет данные для управления конфигурацией процессов ЖЦ ПО в отношении идентификации конфигурации, базовых версий, сообщений о проблемах и управлении изменениями.

7. Хранение и воспроизведение гарантируют, что связанные с программным продуктом данные ЖЦ ПО можно воспроизвести в случае необходимости дублирования, повторного получения, повторного тестирования или изменения программного продукта.
8. Контроль загрузки ПО гарантирует, что Исполняемый Объектный Код и Файлы Компонентов Параметрических Данных, если таковые имеются, загружаются в систему или оборудование с соответствующими защитными мерами.
9. Контроль среды ЖЦ ПО гарантирует, что инструменты, используемые для создания ПО, идентифицируются, контролируются и воспроизводимы.

3. Записи УК ПО

Процесс УК ПО включает в себя регистрацию результатов мероприятий в Записях УКПО.

Под записями УК ПО подразумевается хранение конфигураций в системе контроля версий GitLab.

Задача Jira - сущность, которая имеет название, описание, лицо, которое создало задачу, и лицо, на которое назначили задачу.

Задачи Jira могут находиться в состояниях:

- To do - задачи, которое только были созданы и назначены, и назначенное лицо еще не успело к ним приступить;
- In progress - задачи, которые выполняются прямо сейчас;
- Review - задачи, которые были выполнены исполняющими лицами и отправлены на проверку соответствия внесенным изменениям ответственным за проверку лицам;
- Ready - задача выполнена, все изменения внесены.

Для хранения задач, назначенных на работников используется система Jira.

Для создания и редактирования документов требований, документации используется среда Confluence.

4. Мероприятия УК ПО

В данном разделе представлено описание мероприятий процесса УКПО, проводимых в течение ЖЦ ПО.

4.1. Определение конфигурации

4.1.1. Единица конфигурации и их идентификация

Конфигурация это совокупность всех единиц конфигурации.

Единицы конфигурации:

- Требования к ПО;
- Требования к инфраструктуре;
- Требования к среде разработки ПО;
- Документация к ПО;
- Документ о проблемах;
- Готовая сборка.

4.1.2. Контроль среды ЖЦ ПО

В процессе УК ПО будет использоваться система версионного контроля GitLab, система для созданий и редактирования базы знаний Confluence, система для организации таск-менеджмента Jira.

4.1.2.1 Создание запроса на слияние(merge request)

Merge request(mr) - это запрос на изменения конфигурации, который включает ветку источник, откуда берется само изменение, результирующую ветку, куда должны быть внесены изменения.

Merge request может создать любой работник..

Создание merge request:

1. Зайти в репозиторий проекта;
2. Выбрать справа отдел merge request;
3. Нажать кнопку - New merge request;
4. Выбрать необходимые ветки проекта;
5. Нажать на кнопку - Compare branches and continue;
6. Выбрать в качестве reviewer'a руководителя проекта;
7. Добавить название и описание;
8. Нажать кнопку - создать MR.

4.1.2.2 Создание документа

В данном разделе описан процесс создания документа в среде Confluence.

1. Проверить, что страница Confluence, из которой мы хотим создать документ, закрыта для редактирования. Если какая-либо страница все еще открыта, необходимо проверить, используется ли она в открытой задаче текущего спринта. Если страница не используется в открытых задачах, следует обратиться к ответственному разработчику для закрытия страницы.
2. Проверить, что все ссылки на странице в Confluence актуальны и ведут к правильным файлам.
3. Проверить, что все необходимые документы имеют актуальные версии и не были случайно удалены или заменены.
4. Экспортировать необходимые документы.
5. Переименовать файл в соответствии со стандартом по названию версии - “{день месяца} {месяц} {текущий год}-{время генерации}”.

4.2. Архивирование и воспроизведение

4.2.1. Хранение

Все конфигурации хранятся в репозитории системе GitLab, все базовые версии хранятся в этом же репозитории системы GitLab в ветке master, права на редактирование которой имеет человек, ответственный за хранение базовой версии.

4.2.2 Воспроизведение

Все работники, которые участвуют в процессе разработки ПО, имеют доступ к GitLab репозиторию, в котором хранятся все конфигурации этого ПО.

4.3. Базовая версия

Базовая версия - это конфигурация, которая является эталоном на определенный момент времени, на которую разработчики должны ссылаться при внесении своих изменений.

Базовой версией является ветка master системы GitLab.

4.3.1. Процесс сборки базовой версии

1. Создать ветку master в репозитории GitLab;
2. Создать документ требований к ПО;
3. Создать документ документации к ПО;

4. Создать документ требований к инфраструктуре;
5. Создать документ требований к среде разработки ПО;
6. Создать пустой документ о проблемах;
7. Создать папку /documents в текущей ветку и переместить в нее документы из пунктов 1 - 5;
8. Создать папку /src в текущей ветку и переместить в нее код, соответствующий требованиям;
9. Перенести в текущую ветку готовую сборку проекта.

4.4. Регистрация проблем

Проблема появляется при регистрации несоответствии между:

- Входными и выходными данными процессов;
- Данными ЖЦ ПО и стандартами;
- Процессами ЖЦ ПО и планами разработки.

Зарегистрировать проблему может любой участник проекта.

Сообщение о проблеме должно:

- Иметь уникальный идентификатор;
- Перечислять ЕК, содержащих проблему;
- Концентрироваться на одной проблеме;
- Содержать описание проблемы, а не ее решение.

При обнаружении проблемы, любым участником проекта, этот участник обязан создать карточку Jira, записать туда найденную проблему и назначить ее на лицо, ответственное за решение проблем, задаче присваивается состояние **to do**. Этот процесс описывает этап - **создание**.

Как только ответственное лицо увидит назначенную на него задачу о проблемах в состоянии to do, он:

1. Делает запрос на изменение страницы об ошибках в Confluence;
2. При получении разрешения он записывает в документ об ошибках текущую проблему, при отказе задача переходит в этап - **отклонена**, карточка с задачей удаляется;
3. Дописывает карточку задачи в Jira номер проблемы из страницы об ошибках из Confluence;
4. Назначает человека для решения проблемы.

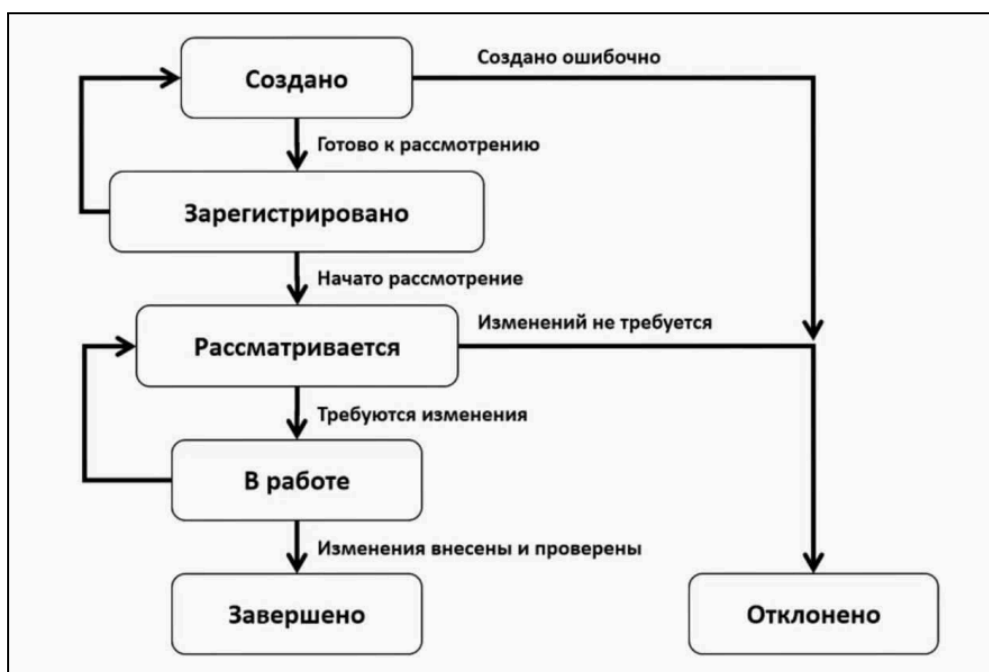


Рисунок 1. ЖЦ сообщения о проблеме

В процессе этого этапа задача переходит в этап - **зарегистрирована**.

Как только человек, который отвечает за решение задачи увидит карточку, то задача переходит в этап **рассмотрения**. Если будет обнаружено, что проблема не имеет правильного обоснования, то задача переходит в этап - **отклонена**, иначе человек начинает решать проблему и она переходит в этап - **в работе**, и человек изменяет статус карточки на **in progress**.

На этапе “**в работе**”, ответственный человек может понять, что он не обладает знаниями для решения проблемы, после чего задача может перейти на этап - **рассматривается** и изменить статус задачи на **to do**, в процессе которого ответственный за решение проблемы человек может спросить о решении проблемы у других людей. Иначе в этом этапе ответственный человек делает запрос на изменение, после чего происходит рассмотрение изменений, при выполнении которого задача переходит в этап - **завершено** и статус карточки изменяется на **ready**.

4.5. Запрос на изменение

Цель запроса на изменение - создание запроса на добавление измененных ЕК в базовую версию.

Запрос на изменение содержит:

- Измененные ЕК;
- Описание изменений.

Запросом на изменение является совокупность:

- Merge request, в котором изменены ЕК;
- Карточка Jira в статусе review.

ЖЦ запроса на изменение

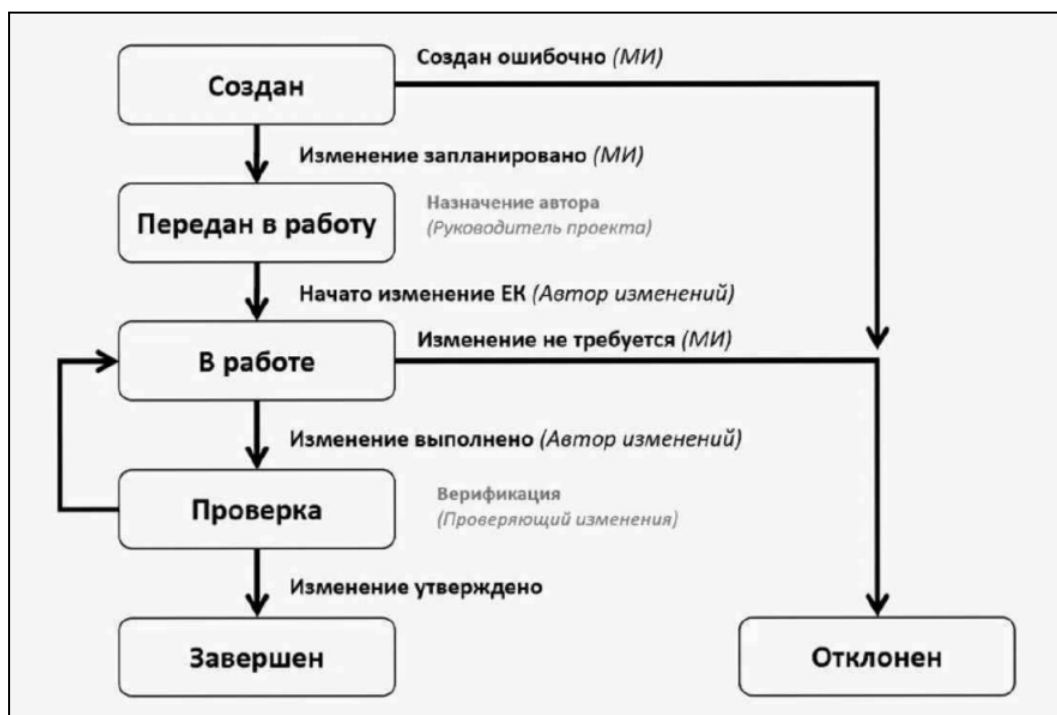


Рисунок 2. ЖЦ запроса на изменение

Процесс запроса начинается с появления необходимости в изменениях, которая может быть обусловлена необходимостью решения проблемы, добавлением нового функционала и т.п. При этом лицо, ответственное за создание задач, создает карточку Jira со статусом to do и записывает в ней описание задачи, после чего назначает на нее на руководителя проекта. Этот этап соответствует блоку - **создан** на диаграмме.

Затем руководитель проекта видит, что на него назначена карточка с запросом на изменение и он назначает на нее исполняющее лицо, этап соответствует - **передан в работу**.

Когда исполняющее лицо приступит выполнению задачи, описанной в карточке, то начинается этап - **в работе**. При понимании, что задача по каким-то причинам не

должна быть выполнена, задача может быть **отклонена**. Процесс работы содержит следующие шаги:

1. Создание ветки из конфигурации, соответствующей добавляемого функционалу
- это может быть базовая версия(ветка master), а может быть и другая конфигурация(другая ветка), в рамках которой создается запрос на изменение;
2. Изменение ЕК в новой ветке;
3. Добавление в ветку документа соответствующего добавленным изменениям;
4. Создание merge request и назначение на его выполнение руководителя проекта;
5. Перенос карточки Jira в статус review.

После создание merge request начинается этап **проверки(рассмотрение изменений)**, в процессе которого руководитель проекта назначает на проверку merge request проверяющего и добавляет его в карточку Jira.

4.6. Рассмотрение изменений

Проверку изменений проводит проверяющее лицо, которого назначит руководитель проекта.

1. В ходе рассмотрения проверяются внесенные изменений в ЕК, на их соответствие документу об изменениях;
2. Если все изменения соответствуют документу об изменениях, то проверяющее лицо делает слияние веток из merge request, и изменяет статус карточки на - ready и удаляет merge request;
3. Если изменения не соответствуют документу об изменениях, то он записывает в соответствующую задаче карточку Jira то, что не соответствует, и изменяет статус карточки на **in progress**.